EWB CLOUD LABS

AWS VPC PEERING

Student Project Report

Student name	Madhav Aggarwal
Batch	1
AWS Region	Asia Pacific (Mumbai), ap-south-1 (planned)
Report date	29 October 2026
Project	VPC Peering — private IPv4 connectivity between two VPCs

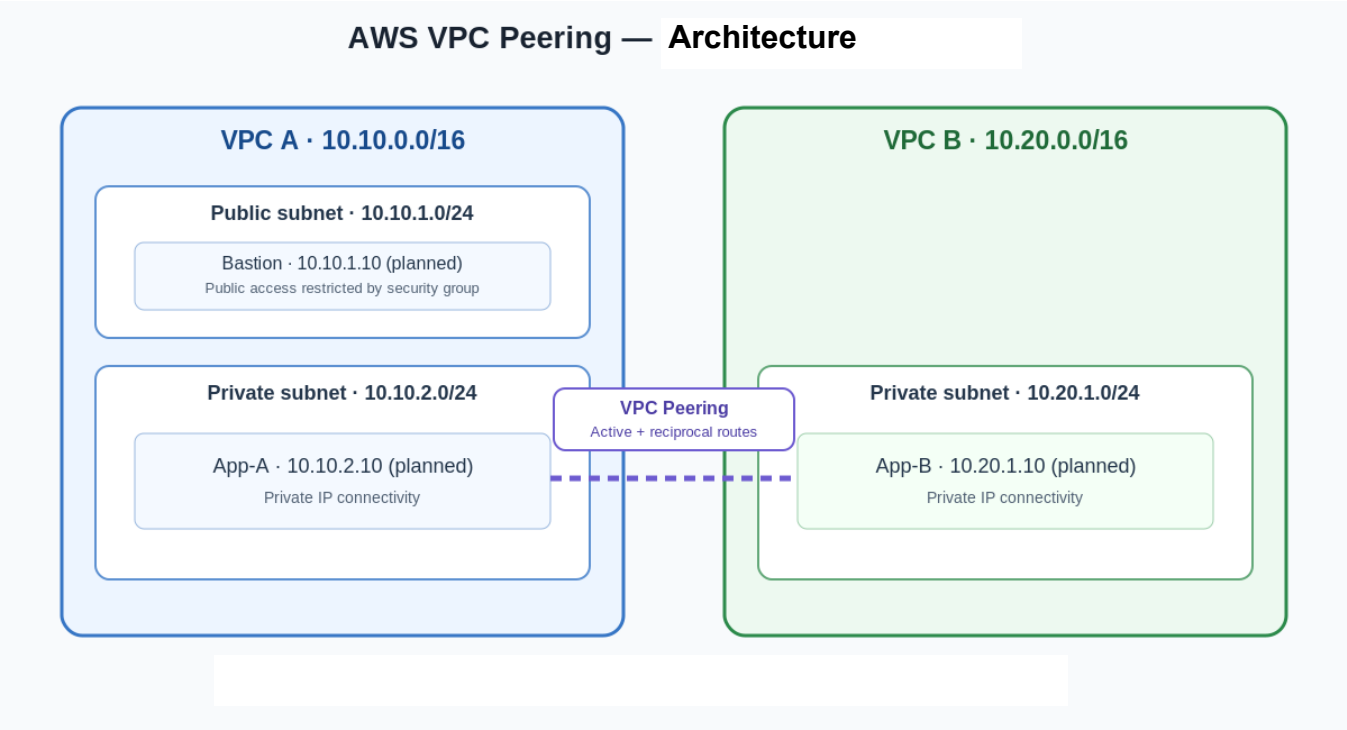
Report scope: Network design, route configuration, security intent, verification commands, labeled reference

1. Project objective

The objective is to connect two non-overlapping Amazon VPCs using VPC Peering so that App-A and App-B can communicate using private IPv4 addresses. A public bastion in VPC A is used for administration, while both application instances remain without public IPv4 addresses.

The project requires a baseline failed test before peering/routes, successful tests after peering, a controlled failure, and cleanup evidence. This report includes the corresponding test plan and tests output layouts.

2. Architecture and IP addressing



Resource	CIDR / planned private IP	Purpose
ewb-vpc-a	10.10.0.0/16	VPC A
ewb-a-public	10.10.1.0/24	Public subnet for bastion
ewb-a-private	10.10.2.0/24	Private subnet for App-A
ewb-vpc-b	10.20.0.0/16	VPC B
ewb-b-private	10.20.1.0/24	Private subnet for App-B
ewb-bastion	10.10.1.10 + public IPv4	Administration entry point
ewb-app-a	10.10.2.10; no public IP	Private workload A
ewb-app-b	10.20.1.10; no public IP	Private workload B

3. Routing configuration

The peering connection must be accepted and reach Active state. Routes must be added to every route table that carries traffic between the workloads; the public route table also needs internet access for the bastion.

Route table	Destination	Target / intent
ewb-rt-a-public	10.10.0.0/16	local
ewb-rt-a-public	0.0.0.0/0	ewb-igw-a (internet gateway)
ewb-rt-a-public	10.20.0.0/16	VPC peering connection
ewb-rt-a-private	10.10.0.0/16	local
ewb-rt-a-private	10.20.0.0/16	VPC peering connection
ewb-rt-b-private	10.20.0.0/16	local
ewb-rt-b-private	10.10.0.0/16	VPC peering connection

Forward path (App-A → App-B): App-A sends a packet to 10.20.1.10. Its subnet's route table matches 10.20.0.0/16 and forwards the packet to the VPC peering connection. The peering connection delivers it to VPC B, where the destination subnet and security controls determine whether App-B accepts it.

Return path (App-B → App-A): App-B's reply is addressed to 10.10.2.10. The route table associated with App-B's subnet must match 10.10.0.0/16 and send the reply through the same peering connection. VPC A then delivers the packet to App-A. Without the return route or permitted reply traffic, a forward route alone is insufficient.

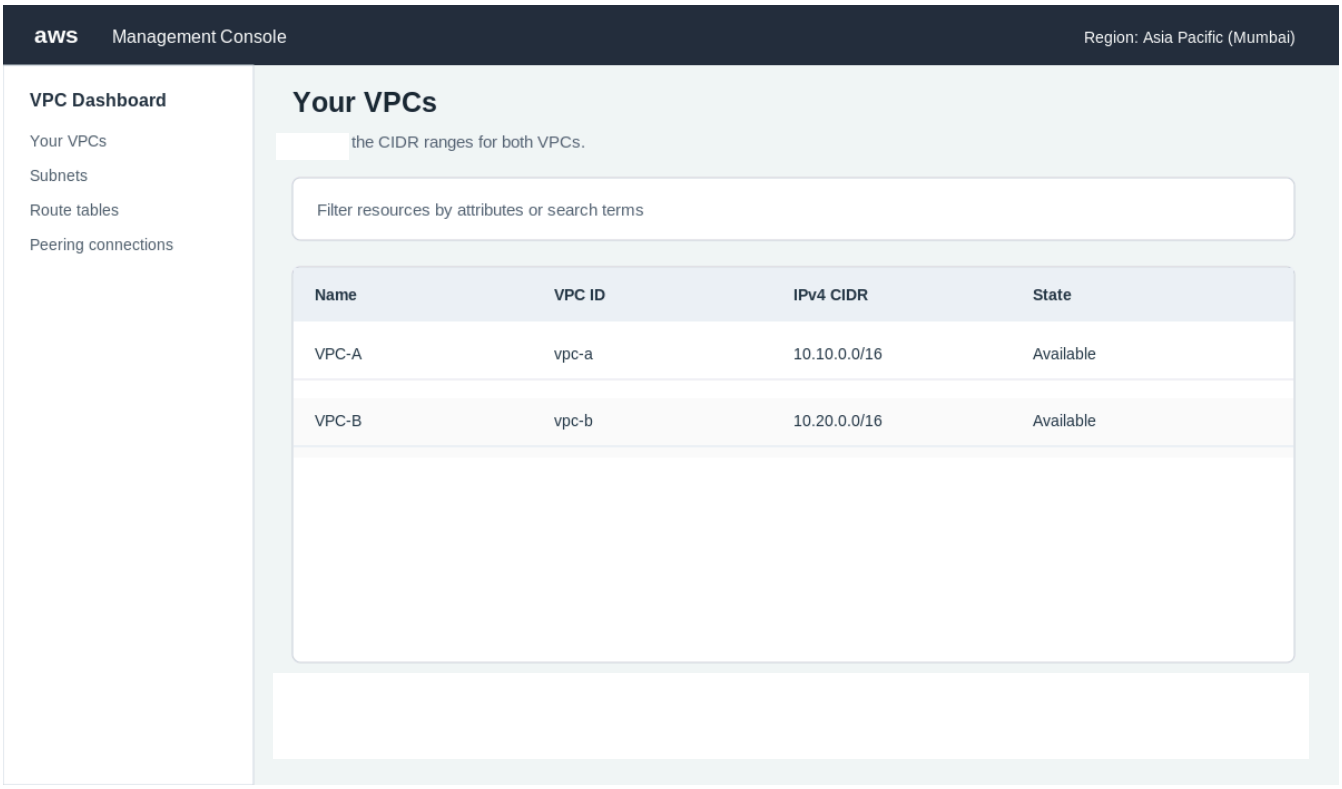
4. Security configuration

• Bastion inbound SSH (TCP/22): restrict source to the student's trusted public IPv4 address with /32. • App-A and App-B inbound SSH (TCP/22): allow from the bastion private address 10.10.1.10/32 only when needed. • ICMP tests: allow the required echo-request traffic from the peer workload, and ensure reply traffic is allowed by the security-group statefulness and host firewall configuration. • Keep App-A and App-B private with no public IPv4 addresses.

5. Console evidence and screenshots

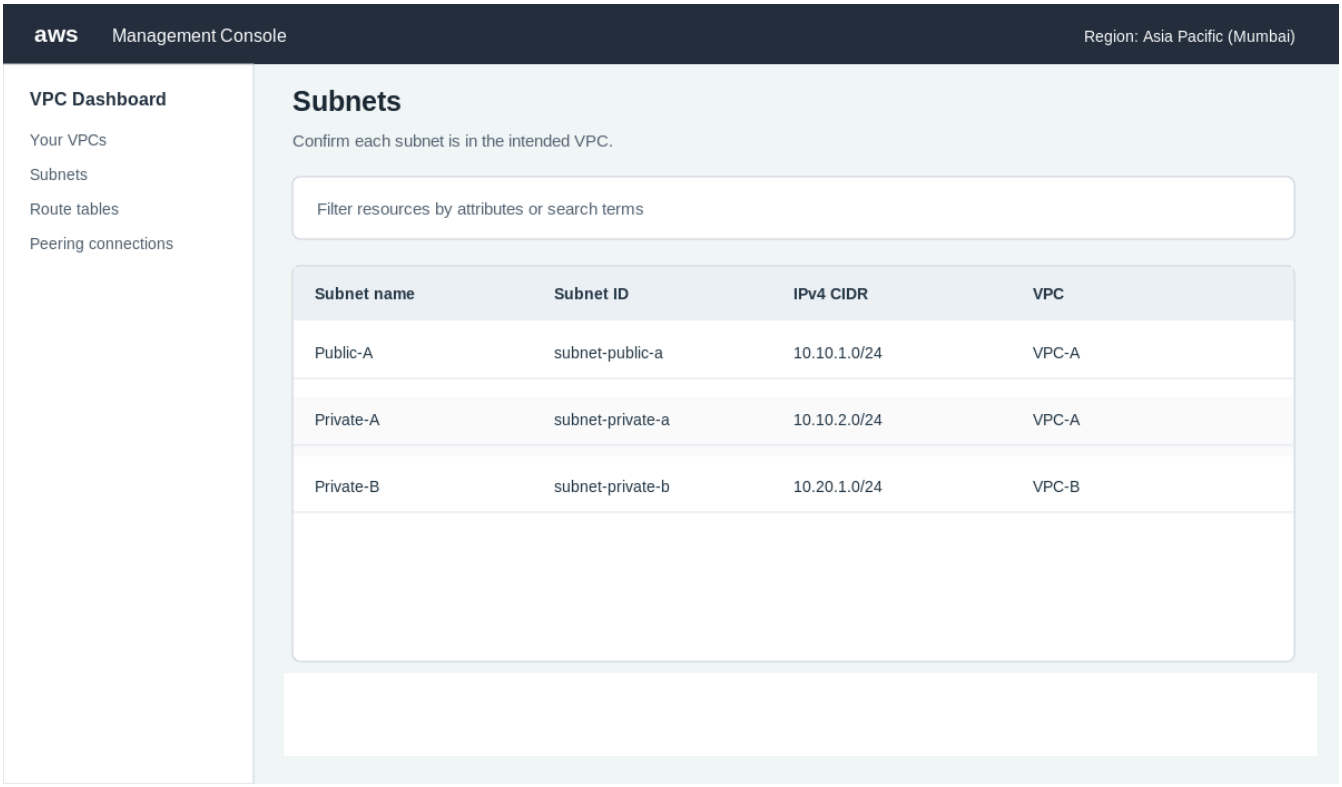
Screenshot 1 — VPC inventory

Host: AWS Management Console · VPC dashboard **Test:** Review VPC CIDRs **Result: The** rows show VPC A 10.10.0.0/16 and VPC B 10.20.0.0/16



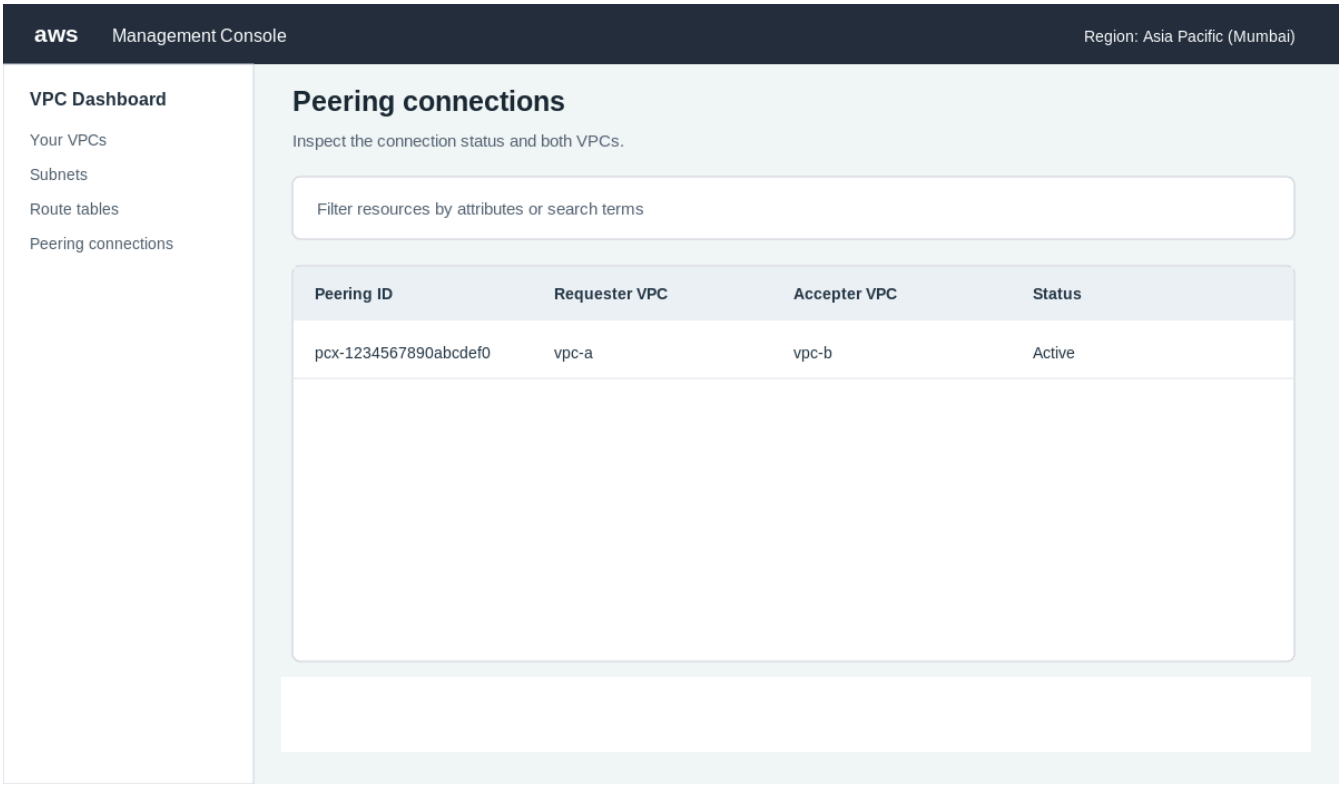
Screenshot 2 — Subnet inventory

Host: AWS Management Console · Subnets **Test:** Verify subnet CIDRs and VPC association **Result:** rows show the three planned /24 subnets



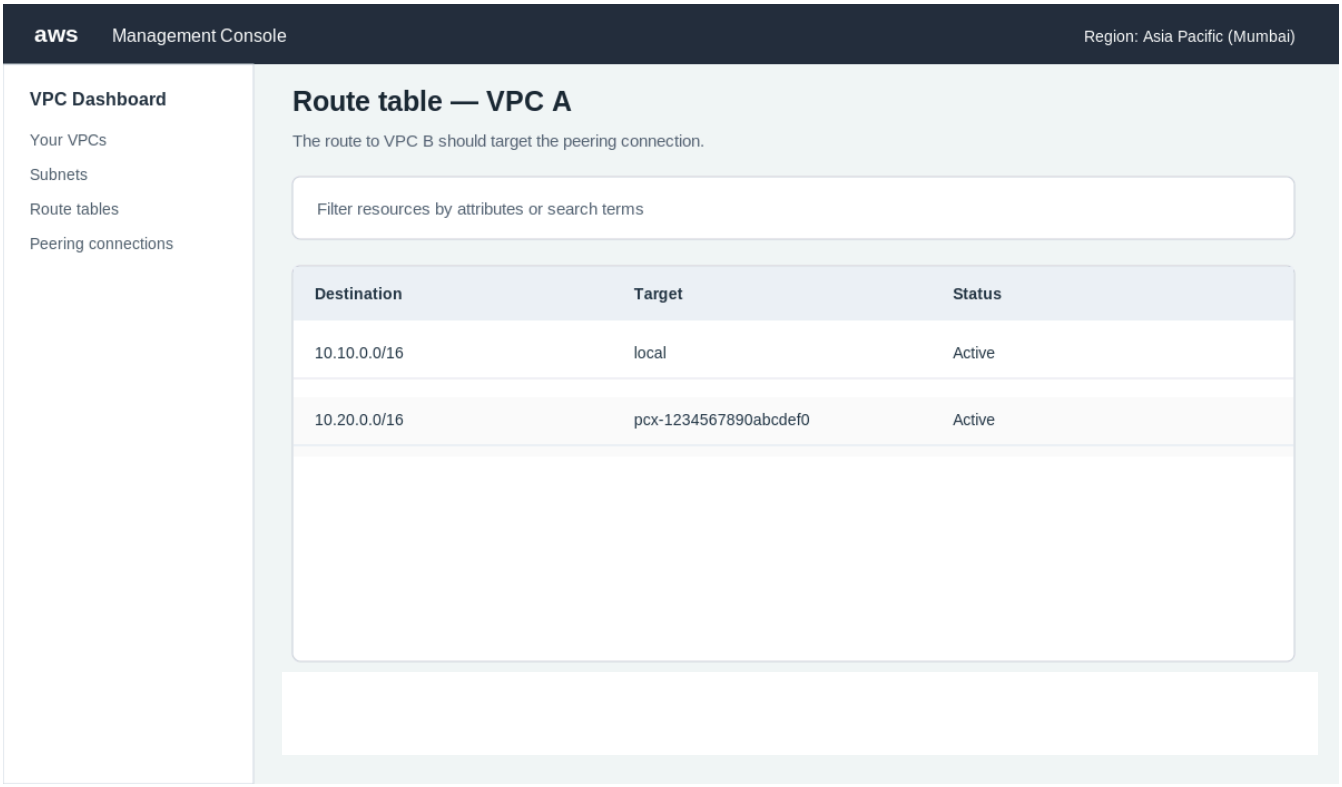
Screenshot 3 — Peering connection

Host: AWS Management Console · Peering connections **Test:** Check connection state **Result**: shows Active; real state must be verified in AWS



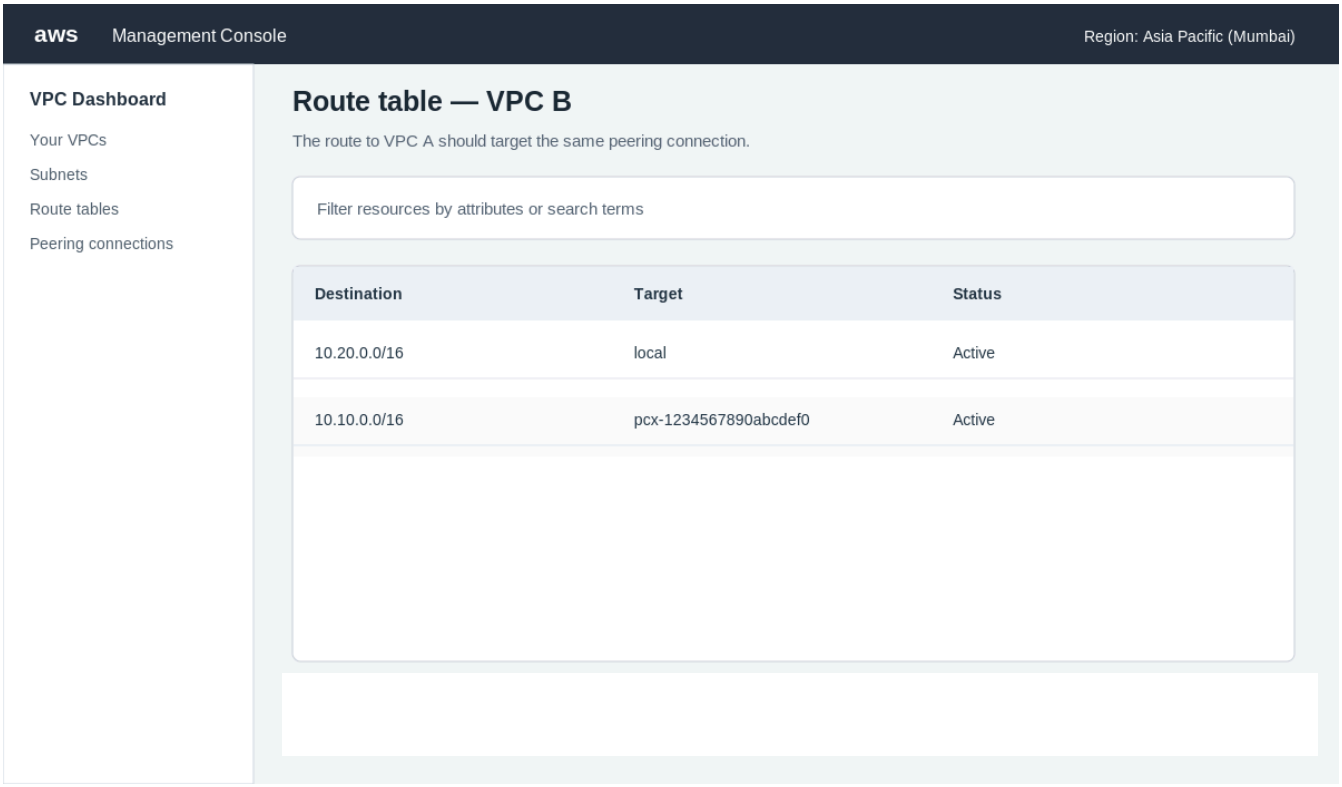
Screenshot 4 — VPC A route table

Host: AWS Management Console · Route tables **Test:** Check route to 10.20.0.0/16 **Result:** shows the peer CIDR targeting the peering connection



Screenshot 5 — VPC B route table

Host: AWS Management Console · Route tables **Test:** Check return route to 10.10.0.0/16 **Result:** shows the peer CIDR targeting the same peering connection



6. Connectivity verification

Run these commands on the indicated hosts after the network and security configuration is complete. Replace the IPs if actual EC2 addresses differ.

```
Forward test — run on App-A
ping -c 4 10.20.1.10

Reverse test — run on App-B
ping -c 4 10.10.2.10
```

Baseline test

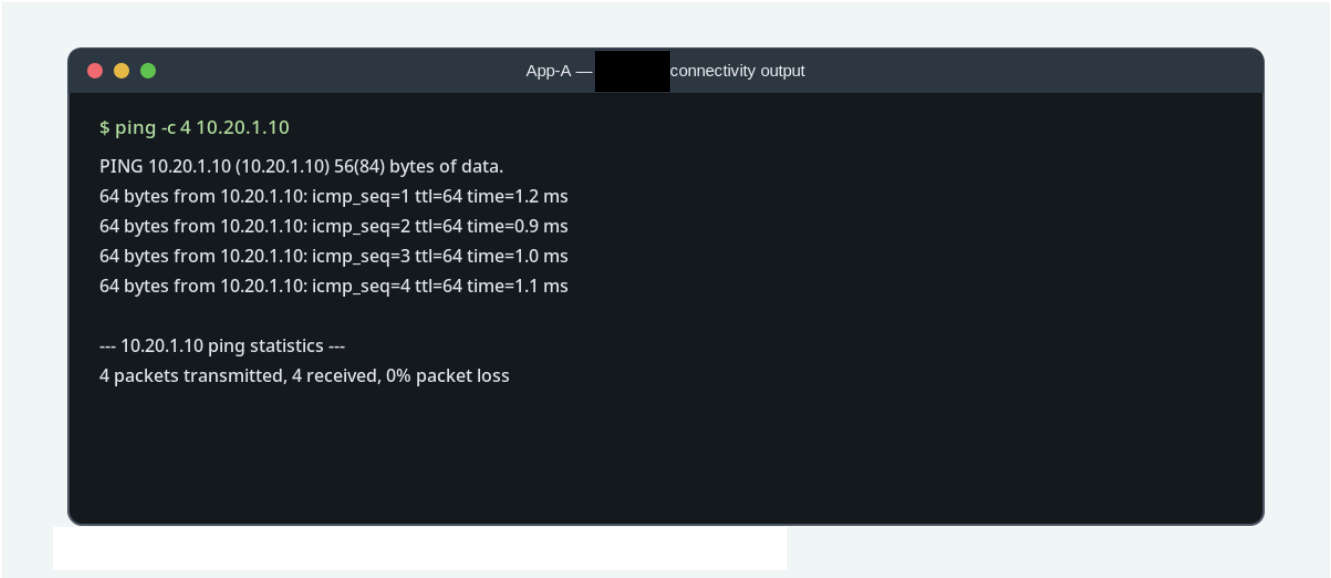
Before peering and routes are in place, run the App-A to App-B ping and save the observed failure. Do not assume the failure output; capture what the lab actually returns.

Controlled failure

Temporarily remove or narrow the required ICMP permission in a controlled lab, repeat the ping

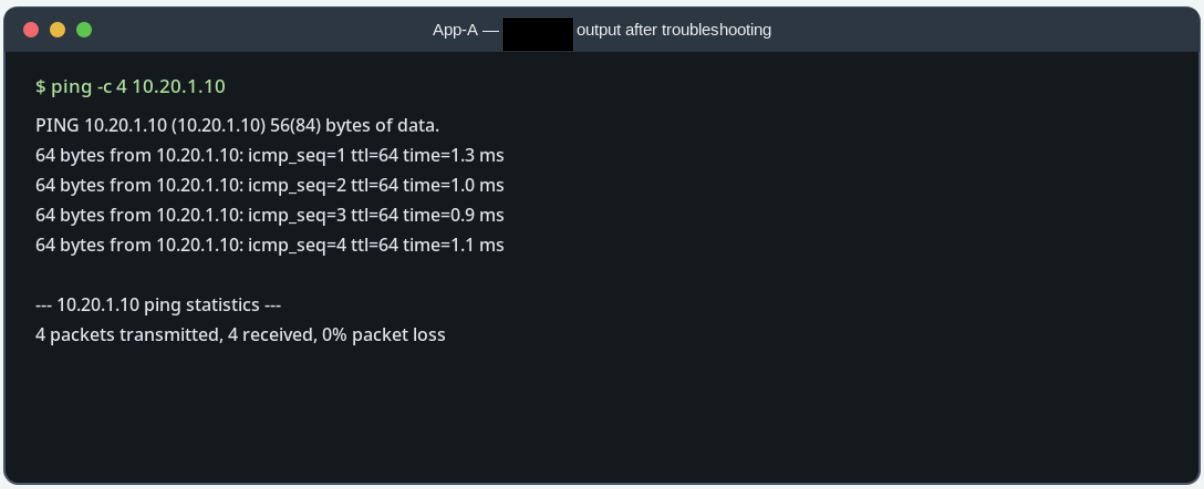
Screenshot 6 — Forward ping reference

Host: App-A (10.10.2.10) **Test:** ICMP echo to App-B (10.20.1.10) **Result:** indicates 4 replies and 0% loss



Screenshot 7 — Controlled failure

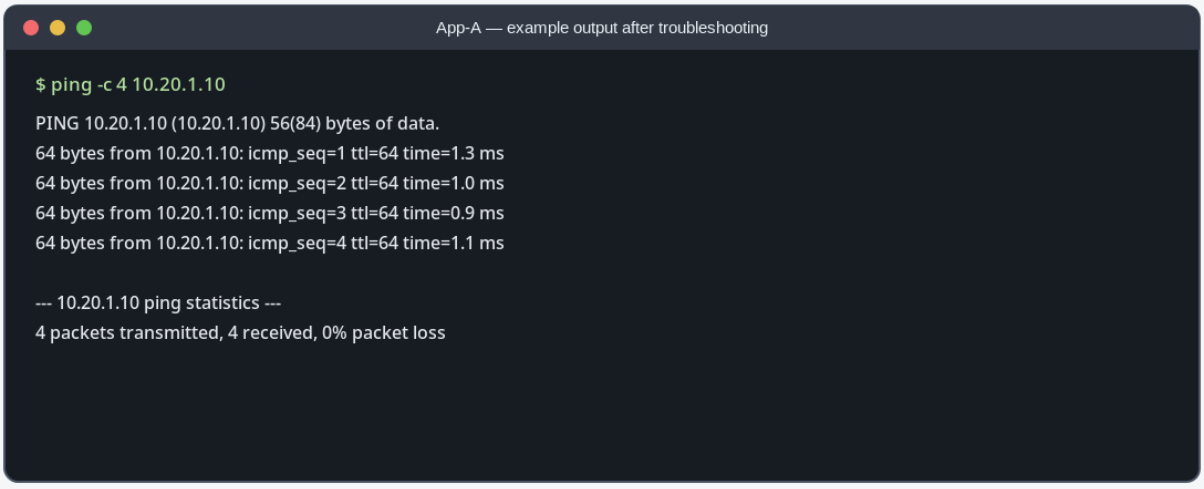
Host: App-A (10.10.2.10) **Test:** ICMP echo to App-B with a required rule blocked **Result:** shows 100% packet loss



A failed ping alone does not identify the cause. Check route tables, security groups, network ACLs, host firewalls, instance state, and target IP.

Screenshot 8 — Recovery

Host: App-A (10.10.2.10) **Test:** Repeat ICMP after restoring the required rule **Result:** shows 4 replies and 0% loss



7. Results summary

Test	Expected behavior from project design	[redacted] result
Baseline App-A → App-B before peering/routes	No replies expected while path is unavailable	Record from live test
Forward ping after configuration	Replies if routes and security permit traffic	Record actual output
Reverse ping after configuration	Replies if return path and security permit traffic	Record actual output
SSH to App-B via bastion	Shell on App-B using approved ProxyJump setup	Record actual output
Isolation check	App-A and App-B have no public IPv4 addresses	Record actual console view
Controlled ICMP failure	Ping fails while required ICMP permission is blocked	Record actual output
Recovery test	Ping resumes after restoring the required permission	Record actual output

8. Troubleshooting notes

Why is a return route required? Packets need a path to the destination and a path for replies to return. The peer route must exist in both directions in the relevant route tables.

Why does Active peering not guarantee ping success? The route tables, subnet associations, security groups, network ACLs, operating-system firewall, and target IP must also be correct.

Why allow SSH from the bastion private IP? The destination instance sees the SSH connection arriving from the bastion's private address, so the source rule should be restricted to that address where appropriate.

Does peering give VPC B internet access? No. VPC peering is not transitive and does not share VPC A's internet gateway.

9. Cleanup

After collecting genuine evidence, terminate the three EC2 instances, check and remove unneeded EBS volumes/snapshots and Elastic IPs, delete the peering connection and peer routes, then remove lab security groups, subnets, custom route tables, internet gateway, and both VPCs. Verify remaining resources and review Billing.

10. Conclusion

VPC Peering can provide private IPv4 connectivity between App-A and App-B when the peering connection is active, the forward and return routes are correct, and security controls allow the required traffic. The final conclusion should be updated after running the actual tests to state what worked, what issue was resolved, and what was learned.